Day 120 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 2 reference

Watch: Hierarchical clustering - explained (TileStats) | Hierarchical Clustering & Dendrograms — Single, Complete, Average & Ward Linkage (Naive Statistician)

`day120_hierarchical_clustering.py`, segment by segment, in the order `main()` uses things. The merge loop, the cut, the cophenetic heights and the dendrogram drawing are new. Day 119's sqdist, ari, standardise and kmeans are imported unchanged; SciPy appears only as an outside check. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: D = the table of cluster distances · Z = the merge history, one row (a, b, height, size) per merge · n_i = a cluster's size · i, j = the two clusters merged, k = any other

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day119_k_means as d119
sqdist, ari, standardise = d119.sqdist, d119.ari, d119.standardise
LINKS = ["single", "complete", "average", "ward"]
```

- `with contextlib.redirect_stdout(io.StringIO()):` — Day 119's import would print nothing anyway; this keeps it quiet if it ever did.
- `sqdist, ari, standardise = d119.sqdist, d119.ari, d119.standardise` — three names taken from Day 119 in one line: tuple assignment.

Segment 1 — Agglomerative clustering

agglomerate: the loop

CODE

```
def agglomerate(X, method):
    n = len(X); D = np.sqrt(np.maximum(sqdist(X, X), 0))
    if method == "ward":
        D = D ** 2
    D[np.arange(n), np.arange(n)] = np.inf
    size = np.ones(n); ids = np.arange(n); alive = np.ones(n, bool); Z = []
    for step in range(n - 1):
        M = np.where(alive[:, None] & alive[None, :], D, np.inf)
        i, j = np.unravel_index(np.argmin(M), M.shape); i, j = min(i, j), max(i, j)
        d = M[i, j]; ni, nj = size[i], size[j]
```

- `D[np.arange(n), np.arange(n)] = np.inf` — the diagonal set to infinity, so the minimum is never a cluster with itself.
- `alive[:, None] & alive[None, :]` — an n x n mask: True only where both row and column are clusters still in play.
- `np.unravel_index(np.argmin(M), M.shape)` — argmin gives a position in the flattened table; unravel_index turns it back into (row, column).

- `i, j = min(i, j), max(i, j)` — both sides are computed before assigning, so this swaps safely.

agglomerate: the Lance-Williams update

MATH

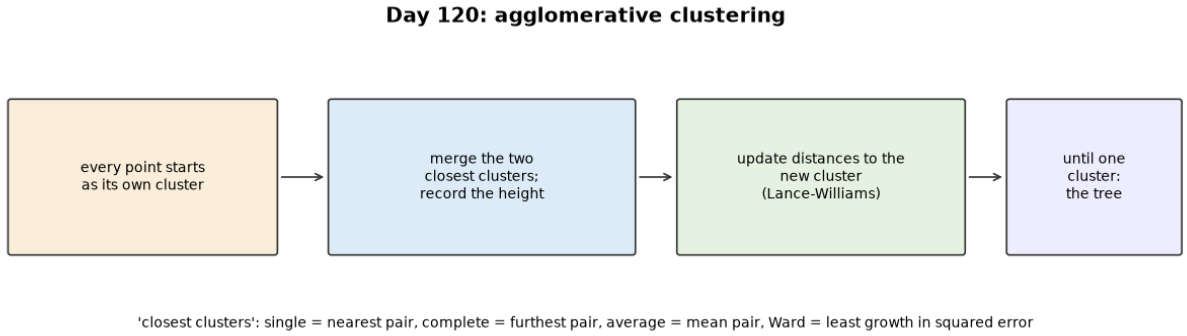
Ward: $d(i \cup j, k) = ((n_i + n_k)\, d_{ik} + (n_j + n_k)\, d_{jk} - n_k\, d_{ij}) / (n_i + n_j + n_k)$, *on squared distances*

CODE

```
Z.append((min(ids[i], ids[j]), max(ids[i], ids[j]), np.sqrt(d) if method == "ward" else d, ni +
    nj))
nk = size
if method == "single":
    new = np.minimum(D[i], D[j])
elif method == "complete":
    new = np.maximum(D[i], D[j])
elif method == "average":
    new = (ni * D[i] + nj * D[j]) / (ni + nj)
else:
    new = ((ni + nk) * D[i] + (nj + nk) * D[j] - nk * D[i, j]) / (ni + nj + nk)
D[i, :] = new; D[:, i] = new; D[i, i] = np.inf
alive[j] = False; size[i] = ni + nj; ids[i] = n + step
```

- `np.minimum(D[i], D[j])` — element by element: the merged cluster's distance to every other cluster, for single linkage.
- `(ni * D[i] + nj * D[j]) / (ni + nj)` — average linkage: the old distances weighted by cluster size.
- `D[i, :] = new; D[:, i] = new` — the merged cluster reuses row and column i; row j is retired.
- `ids[i] = n + step` — SciPy's numbering: merges are named n, n + 1, ... in order.

SYSTEM DIAGRAM



Merge, update, repeat.

Segment 2 — Reading the tree

cut: labels for k clusters

CODE

```
def cut(Z, n, k):
    parent = np.arange(2 * n - 1)
    for s, (a, b, _, _) in enumerate(Z[:n - k]):
        parent[int(a)] = parent[int(b)] = n + s
    def root(x):
        while parent[x] != x:
            x = parent[x]
        return x
    r = np.array([root(i) for i in range(n)])
    return np.unique(r, return_inverse=True)[1]
```

- `parent = np.arange(2 * n - 1)` — n points plus n - 1 merges; each node starts as its own parent.
- `parent[int(a)] = parent[int(b)] = n + s` — chained assignment: both children now point to the merge that joined them.
- `def root(x):` — a function defined inside a function: it can read parent directly.
- `Z[:n - k]` — replaying all but the last k - 1 merges leaves k clusters.

cophenetic and the leaf order

MATH

$$\textit{cophenetic correlation} = \textit{corr}(|x_a - x_b|, \textit{merge height joining a and b}), \textit{over all pairs}$$

CODE

```
def cophenetic(Z, n):
    members = {i: [i] for i in range(n)}; C = np.zeros((n, n))
    for s, (a, b, h, _) in enumerate(Z):
        A, B = members.pop(int(a)), members.pop(int(b))
        C[np.ix_(A, B)] = h; C[np.ix_(B, A)] = h
        members[n + s] = A + B
    return C
def coph_corr(X, Z):
    n = len(X); D = np.sqrt(np.maximum(sqdist(X, X), 0)); C = cophenetic(Z, n); iu =
        np.triu_indices(n, 1)
    return float(np.corrcoef(D[iu], C[iu])[0, 1])
def leaf_order(Z, n):
    kids = {n + s: (int(a), int(b)) for s, (a, b, _, _) in enumerate(Z)}
    def walk(x):
        return [x] if x < n else walk(kids[x][0]) + walk(kids[x][1])
    return walk(2 * n - 2)
```

- `members.pop(int(a))` — removes a cluster from the dictionary and returns its points.
- `C[np.ix_(A, B)] = h` — np.ix_ selects every (row in A, column in B) pair: all cross pairs meet at height h.
- `np.triu_indices(n, 1)` — the pairs above the diagonal: each pair once.
- `walk(kids[x][0]) + walk(kids[x][1])` — recursion: a node's leaves are its left child's leaves, then its right child's.

Segment 3 — Drawing

draw_dendrogram: each merge as an upside-down U

CODE

```
def draw_dendrogram(ax, Z, n, labels=None, colour_k=None):
    order = leaf_order(Z, n); xpos = {leaf: i for i, leaf in enumerate(order)}; ypos = {i: 0.0 for
        i in range(n)}
    groups = cut(Z, n, colour_k) if colour_k else None; cmap = plt.get_cmap("tab10")
    for s, (a, b, h, _) in enumerate(Z):
        a, b = int(a), int(b); xa, xb = xpos[a], xpos[b]
        same = None
        if groups is not None:
            ga = {groups[l] for l in (leaf_order_sub(Z, n, a))}; gb = {groups[l] for l in
                leaf_order_sub(Z, n, b)}
            same = ga.pop() if len(ga) == 1 and ga == gb else None
        c = cmap(same % 10) if same is not None else "#555555"
        ax.plot([xa, xa, xb, xb], [ypos[a], h, h, ypos[b]], color=c, linewidth=0.8)
        xpos[n + s] = (xa + xb) / 2; ypos[n + s] = h
    ax.set_xticks([]); ax.set_xlim(-1, n)
```

- `xpos = {leaf: i for i, leaf in enumerate(order)}` — leaves are placed left to right in tree order, so branches never cross.
- `ax.plot([xa, xa, xb, xb], [ypos[a], h, h, ypos[b]])` — four points, one line: up from child a, across at height h, down to child b.
- `xpos[n + s] = (xa + xb) / 2` — the merged node sits midway between its children, ready for the next merge above it.
- `ga.pop() if len(ga) == 1 and ga == gb else None` — a branch is coloured only if everything under it falls in one group of the cut.

leaf_order_sub: a cache

CODE

```
_SUB = {}
def leaf_order_sub(Z, n, node):
    key = (id(Z), node)
    if key not in _SUB:
        if node < n:
            _SUB[key] = [node]
        else:
            a, b = Z[node - n][:2]; _SUB[key] = leaf_order_sub(Z, n, int(a)) + leaf_order_sub(Z,
                n, int(b))
    return _SUB[key]
```

- `key = (id(Z), node)` — id() is the object's identity, so two different trees never share entries.
- `if key not in _SUB:` — compute once, then reuse: drawing asks for the same subtrees many times.

Segment 4 — The experiments

demo_check: breaking ties with noise

CODE

```
D = np.sqrt(np.maximum(sqdist(X, X), 0))[np.triu_indices(len(X), 1)]
print(f"iris has {len(D) - len(np.unique(np.round(D, 10)))} tied distances (iris measurements are
    rounded to 0.1 cm); with ties, complete linkage can build a different tree")
Xj = X + np.random.RandomState(0).normal(scale=1e-6, size=X.shape)
print("the same check after adding noise of size 1e-6 to break the ties: largest gap " + ",
    ".join(
    f"{m} {np.abs(np.sort(agglomerate(Xj, m)[:, 2]) - np.sort(linkage(Xj, m)[:, 2])).max():.1e}"
        for m in LINKS))
```

- `len(D) - len(np.unique(np.round(D, 10)))` — pairs minus distinct values: how many distances repeat an earlier one.
- `normal(scale=1e-6, size=X.shape)` — tiny noise: distances change in the sixth decimal, enough to break every tie.
- `np.sort(...[:, 2])` — heights compared in sorted order, so merges taken in a different order still line up.

demo_gap: cutting at the largest gap

CODE

```
Z = agglomerate(X, "ward"); h = Z[:, 2]; gaps = np.diff(h)[-15:]
k = int(len(gaps) - np.argmax(gaps) + 1); lab = cut(Z, len(X), k)
```

- `np.diff(h)[-15:]` — the differences between successive heights, among the last merges.
- `len(gaps) - np.argmax(gaps) + 1` — the last gap (index len - 1) means undoing one merge: 2 clusters; an earlier version forgot the + 1.

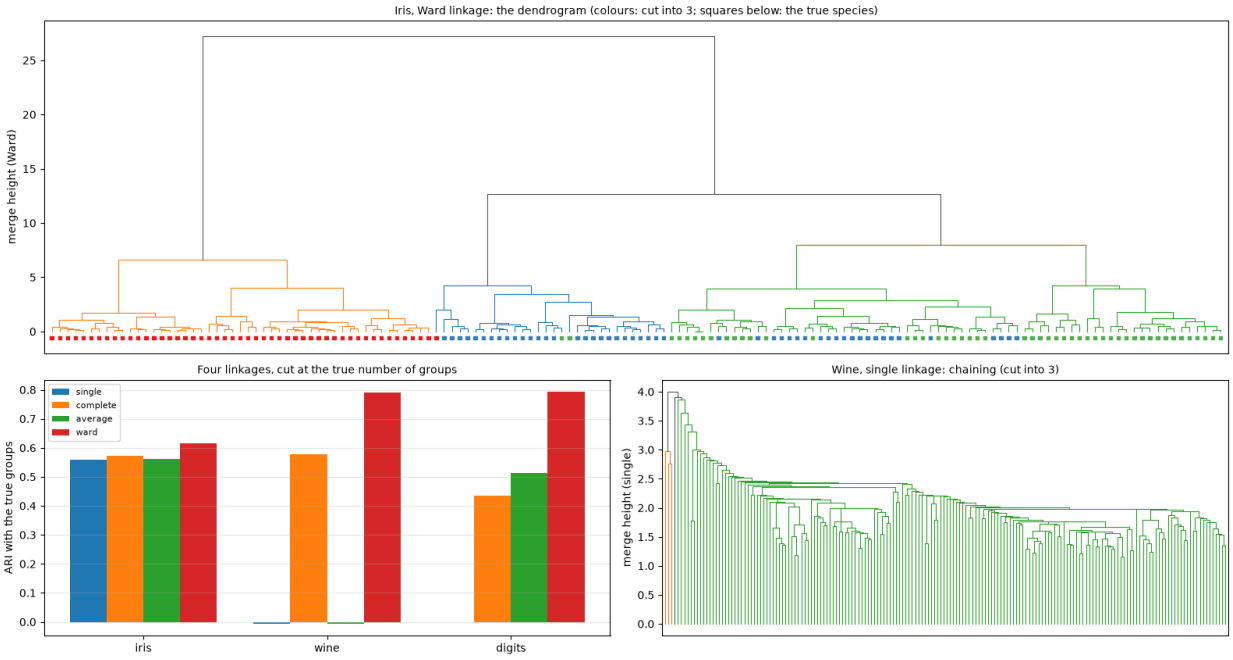
demo_cost: timing the tree

CODE

```
t0 = time.perf_counter(); agglomerate(X[:n], "ward"); t = time.perf_counter() - t0
print(f"  {n:>5} images: {t:.2f} s, and a distance table of {n * n * 8 / 1e6:.1f} MB")
```

- `time.perf_counter()` — a high-resolution clock; the difference of two readings is the elapsed time.
- `n * n * 8 / 1e6` — n x n numbers of 8 bytes each, in megabytes.

REAL GRAPH



Real graph from the script: the iris dendrogram, the four linkages, and single linkage on wine.

Quick Syntax Glossary — new or reinforced constructs

- `np.unravel_index(i, shape)` — flat position to (row, column).
- `a[:, None] & a[None, :]` — an outer AND: a pairwise mask.
- `x = y = value` — chained assignment.
- `np.ix_(A, B)` — every (row, column) pair from two lists.
- `dict.pop(key)` — remove and return.
- `def inner(): ... inside def` — a nested function that reads the outer one's variables.

End of Day 120 syntax reference. For the experiments and what they showed, see the Day 120 complete guide.